# ADA BOOST

A visual learning note on how small rules become one confident classifier.

**Core idea**

**Train weak learners one after another. Each round pays more attention to examples the earlier rounds got wrong.**

1. **Start equal**
   Every row matters equally.

2. **Spot mistakes**
   Use weighted error, not raw accuracy.

3. **Turn up focus**
   Misclassified rows gain weight.

4. **Vote together**
   Reliable learners get louder votes.

READ LEFT TO RIGHT, ROUND BY ROUND

Includes: stumps, error, alpha, weight updates, resampling, and final voting.

# 02 The mental model

AdaBoost does not make one brilliant learner. It orchestrates a sequence of simple ones.


**A. begin** Give all N examples the same weight: 1 / N.

**B. fit** Choose a weak learner that minimizes weighted mistakes.

**C. score** Compute error e, then learner strength alpha.

**D. refocus** Increase weights on mistakes; normalize; repeat.

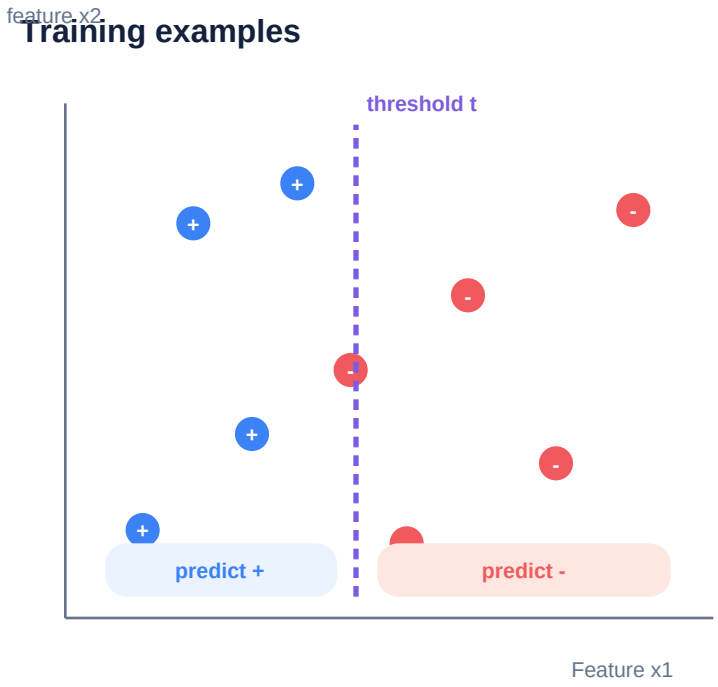
**The essential loop**

**easy examples -> weak learner learns them -> hard examples become more visib**

The model progressively directs attention where the current ensemble is weakest.

# 03 Weak learner: a decision stump

A stump asks one short question, such as "is x less than a threshold?"




**A stump could say:**

**if x1 < t:**

**predict +1**

**otherwise:**

**predict -1**

### Why "weak" is okay

A stump may be only slightly better than random. AdaBoost makes its errors useful information for the next round.

### Choose the stump with minimum weighted error

**Error is the total weight of wrong examples - not simply the number of wrong examples.**

So a single high-weight miss can matter more than several low-weight misses.

# 04 Error becomes influence

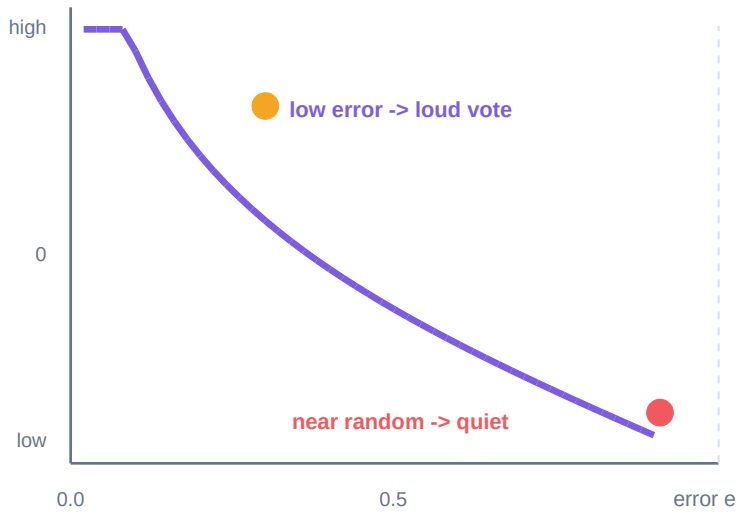
A learner that makes fewer weighted mistakes earns a stronger vote.

**Weighted error**

## e = sum of weights on misclassified examples

Learner strength: alpha = 1/2 ln((1 - e) / e)

### Error e vs. alpha (vote strength)




### Reading the curve

**e = 0.10**
**alpha = 1.10**
very strong

**e = 0.30**
**alpha = 0.42**
useful

**e = 0.49**
**alpha = 0.02**
almost silent

# 05 Weight update: where attention moves

After a round, misclassified examples become more important; then all weights are normalized.

### Before round 1

Every row begins equally.


### After a weak learner

The two missed rows now dominate.


## The update rule

**w(i) <- w(i) x exp(-alpha * y(i) * h(x(i)))**

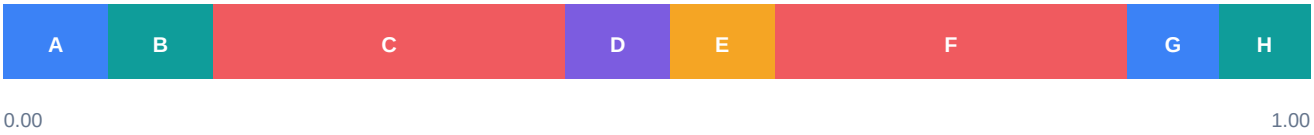
Correct prediction: y*h = +1 -> weight shrinks.

Wrong prediction: y*h = -1 -> weight grows.

# 06 Resampling: turning weights into chances

A common training view: draw rows in proportion to their current weights.

## Probability ruler




**C and F own bigger ranges, so they are drawn more often.**

## Imagine four random draws

| | | |
|---|---|---|
| random number .06 | → | draw A |
| random number .31 | → | draw C |
| random number .49 | → | draw C |
| random number .79 | → | draw F |

## Why it helps

The learner sees difficult cases again and again. Easy cases still appear, but less often.

**Same data.**

**Different emphasis.**

# 07 The full boosting journey

Each round changes the dataset the next stump effectively sees.

ROUND 1

**stump sees equal weights**

misses: C, F


ROUND 2

**C and F are prominent**

fixes C, misses B


ROUND 3

**B now matters more**

adds a complementary rule

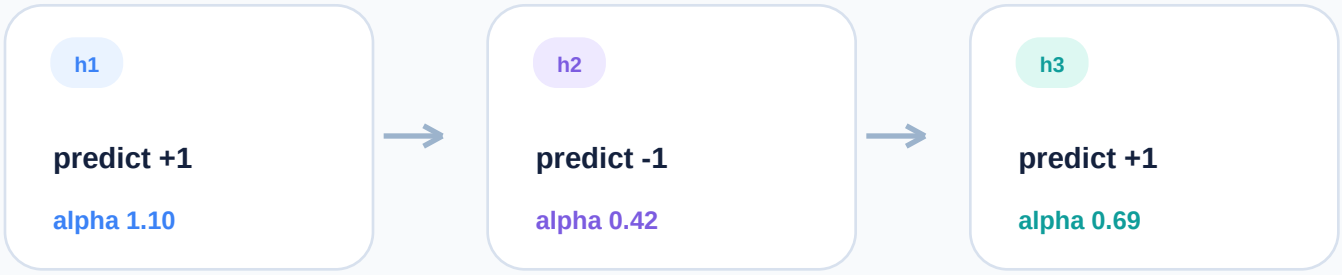

## Iteration checklist

1. Fit h(t) using current weights
2. Measure weighted error e(t)
3. Set alpha(t) from e(t)
4. Increase missed-example weights
5. Normalize and repeat

The outcome is an accumulating committee of learners, not a chain of replacements.

# 08 Final prediction: weighted vote

Every stump speaks; alpha decides how loudly each one speaks.




## F(x) = sign( sum alpha(t) h(t)(x) )

For this example: (+1 x 1.10) + (-1 x 0.42) + (+1 x 0.69) = +1.37

### The vote meter

**Final class: +1**

09

# A tiny worked example and recall sheet

Use this page to rehearse the order of operations before coding AdaBoost.

## Micro example

**N = 8**
initial w = 1/8 = .125
**stump misses 2 rows**
e = .125 + .125 = .25
**compute strength**
alpha = .5 ln(.75/.25) = .55
**update**
wrong rows get multiplied by exp(+.55)

## Three checks before next round

- OK Are the weights normalized to 1?
- OK Is the learner better than random?
- OK Do the high-weight misses make sense?

## AdaBoost in five lines

```
initialize w(i) = 1 / N
for each round t:
  fit weak learner h(t) using w
  compute e(t), then alpha(t)
  upweight mistakes; normalize; vote at the end
```

**Remember: alpha measures a learner; weights measure examples.**

References: Freund & Schapire (1997), Hastie et al. (2009), scikit-learn AdaBoost documentation.

# One complete round - with numbers

Pause at each card, predict what changes, then trace the arrows.

**1. START**

8 rows, each w = .125

**2. STUMP**

misses rows C and F

**3. SCORE**

e = .25, alpha = .55

**4. UPDATE**

C and F receive more mass

## Weight ledger

| row | before | correct? | after normalize |
|---|---|---|---|
| A | 0.125 | yes | 0.083 |
| B | 0.125 | yes | 0.083 |
| C | 0.125 | NO | 0.251 |
| D | 0.125 | yes | 0.083 |
| E | 0.125 | yes | 0.083 |
| F | 0.125 | NO | 0.251 |
| G | 0.125 | yes | 0.083 |
| H | 0.125 | yes | 0.083 |

## Why the math works

The two errors go from 25% of total attention to just over 50% together.

**A later stump is now strongly encouraged to fix C and F.**

### Try it

If the same stump misses C and F again, should its alpha grow or shrink?

**Answer: shrink. Those errors now carry much more weight, so the same rule has a worse weighted error.**

# 11 Use it wisely

AdaBoost is powerful when weak rules complement one another - and fragile when noise dominates.

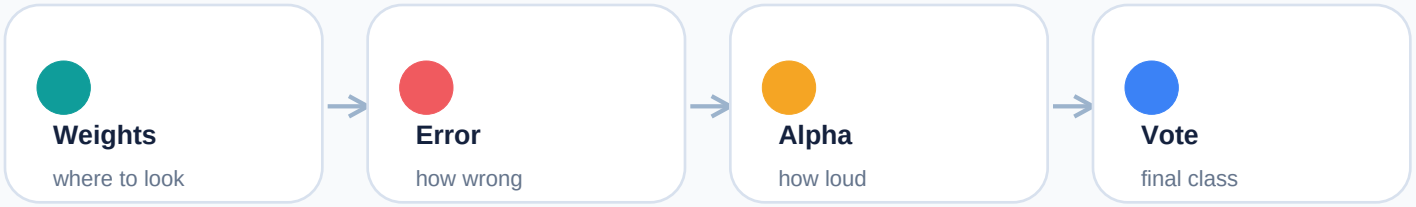
## A good fit

- **Clean signal**
  Simple patterns can combine.
- **Useful features**
  Stumps can split meaningful columns.
- **Moderate noise**
  Hard examples are not mostly label errors.
- **Fast baseline**
  You want interpretable learners quickly.

## Watch out for

- **Label noise**
  The model may chase incorrect labels.
- **Outliers**
  Repeated focus can over-amplify them.
- **Weak learner too strong**
  You lose the intended gradual correction.
- **Error near .50**
  The learner contributes almost no value.

## Fast recall map




**If you remember one sentence: AdaBoost keeps asking, "Which mistakes deserve more attention next?"**

Use the numerical round on page 10 once before you implement it.

# Your dataset story: clean, numeric, and ready

Breast Cancer Wisconsin (Diagnostic): 569 rows, 30 real numeric predictors, and a binary diagnosis target.

## Your starting dataset snapshot

**569** tumor samples

**30** numeric features

**2** diagnosis classes

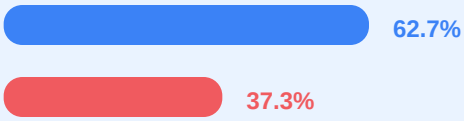
**0** missing values after cleaning

Cleanup: drop id (a record label, not medical signal) and Unnamed: 32 (completely empty).

## Target balance

0 = Benign

1 = Malignant




Moderate imbalance: accuracy is useful, but recall for malignant cases matters.

## Train / test discipline

**455** training rows

**114** final test rows

stratify=y kept the class ratio almost the same in both splits.

**Key notebook move**

```
X = df.drop(columns="diagnosis")
y = df["diagnosis"]
train_test_split(X, y, test_size=0.2,
                 random_state=42, stratify=y)
```

# EDA: turn a table into a story

The point was not to make many plots - it was to understand what AdaBoost could learn from.

## How you read describe()

| | |
|---|---|
| **count** | 455 for each feature -> no missing training values |
| **50%** | median: half the values are below it |
| **25% - 75%** | the middle half of the data |
| **mean > median** | often signals a right-skewed tail |
| **max >> 75%** | extreme values; inspect, do not auto-delete |

## Your examples

**radius_mean**

median 13.34 | max 28.11

**area_mean**

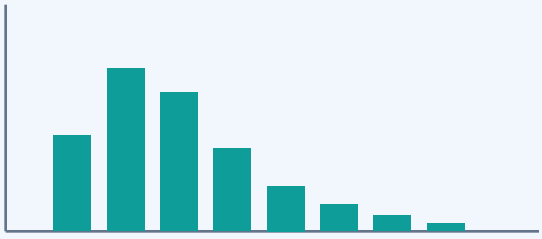
median 546.4 | max 2501

**area_worst**

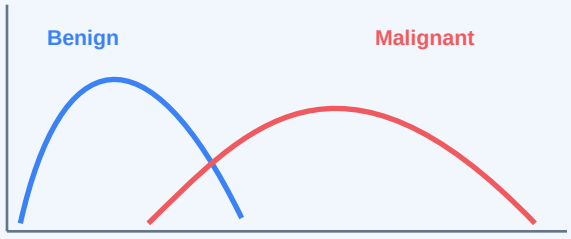
median 683.4 | max 4254

Takeaway: several area-related features have long right tails.

## Histograms and KDE: what you learned



Histogram: many lower values, long tail to the right.




KDE by class: look for shift and overlap, not curve height as patient count.

**Key link to AdaBoost: separation suggests useful thresholds; the tree still chooses the split with the minimum weighted error.**

# EDA: class separation and correlation

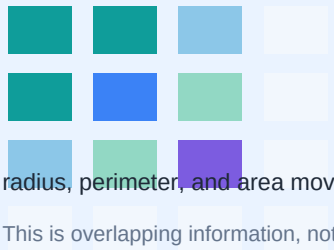
Your plots suggested a consistent medical story: malignant cases tend to be larger and more concavity-related.

## KDE by diagnosis

Helpful features you compared:

- **radius_mean**
  benign lower; malignant shifted right
- **area_mean**
  clear shift toward larger malignant values
- **concavity_mean**
  lower benign values, higher malignant values
- **area_worst**
  especially clear separation

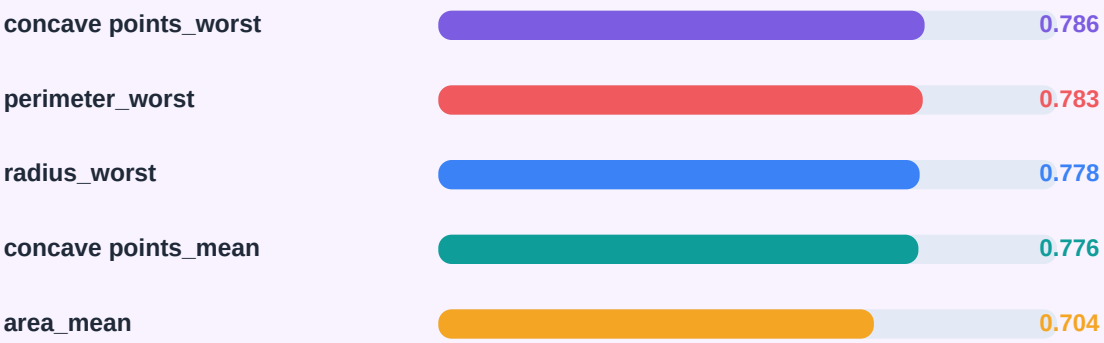
## Correlation matrix



radius, perimeter, and area move together.

This is overlapping information, not a reason to automatically drop columns for tree AdaBoost.

## Target correlation: the strongest individual associations you found




Positive here means higher values were associated with the malignant label (1). Correlation is an EDA clue, not AdaBoost feature importance.

# Baseline AdaBoost: first honest result

Start simple, then measure exactly which cases the model got right and wrong.

**Baseline model - default weak learner is a decision stump**

```
ada = AdaBoostClassifier(
    n_estimators=50, learning_rate=1.0,
    random_state=42)
ada.fit(X_train, y_train)
y_pred = ada.predict(X_test)
```

## Baseline confusion matrix

| | Predicted: Benign | Predicted: Malignant |
|---|---|---|
| Actual benign | 72 | 0 |
| Actual malignant | 2 | 40 |

The 2 false negatives are the critical missed malignant cases.

## Test metrics

| | |
|---|---|
| Accuracy | 98.25% |
| Precision | 100.00% |
| Recall | 95.24% |
| F1 score | 97.56% |

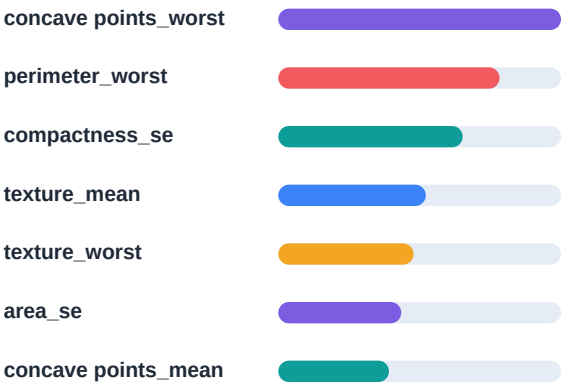
### Reading the result

112 of 114 test cases were correct. Every malignant prediction was correct (precision = 1.0), while 2 of 42 actual malignant cases were missed - which is why recall deserves special attention.

# Feature importance and GridSearchCV

Feature importance shows model use; GridSearchCV tests settings with repeated validation.

## Top AdaBoost features




Overlap with correlation is expected; rankings differ.

## GridSearchCV in one view

**27 settings** x 5 folds = 135 training runs

**n_estimators**
50, 100, 200

**learning_rate**
0.1, 0.5, 1.0

**estimator__max_depth**
1, 2, 3

Select the settings with the highest mean malignant recall.

## Your tuning result and the lesson it taught

**Best CV recall: 97.36%**

best params: max_depth=2 | learning_rate=0.5 | n_estimators=50

**Baseline**

Accuracy 98.25% Recall 95.24%
F1 97.56%

**Tuned**

Accuracy 97.37% Recall 92.86%
F1 96.30%

**Higher CV recall did not guarantee a higher test result. The untouched test set is the final reality check.**

# 17 My mistakes and corrections

These are not failures - they are the exact questions that built a correct mental model.

1. **KDE crossing = stump threshold**
   No. It is a visual clue. The stump tests real candidates and picks minimum weighted error.

2. **Accuracy labels called recall**
   The score is whatever scoring= says. scoring="accuracy" -> label the outputs accuracy.

3. **estimator was confusing**
   Inside AdaBoost, estimator means the repeated base learner / weak model.

4. **OHE for the target**
   For this binary target, map B -> 0 and M -> 1. OHE is mainly for categorical inputs.

5. **baseline ada variable overwritten**
   Recreate baseline_ada explicitly before comparing it with a tuned model.

6. **Better CV means better test result**
   No. CV estimates performance on training folds; the final test score can be different.

7. **Correlation = feature importance**
   No. Correlation is association; importance is use by the fitted model.

8. **Skewed features must be scaled**
   Not for tree stumps. Threshold splits are largely scale-insensitive.

**Best habit: write the metric, data split, model definition, and result next to every comparison.**

# End-to-end checklist and revision map

Use this as the one-page memory sheet before explaining or rebuilding the project.

1. **Load + inspect**
   shape, dtypes, target
2. **Clean**
   drop id / empty column; check duplicates
3. **Encode + split**
   B=0, M=1; stratify; protect test set
4. **EDA**
   describe, distributions, class KDE, correlation
5. **Baseline**
   stumps -> metrics -> confusion matrix
6. **Interpret**
   feature importance and error types
7. **Tune**
   GridSearchCV with an intentional scoring metric
8. **Final report**
   compare fairly; explain CV vs test result

## AdaBoost

Weak learners learn in rounds. Missed rows gain weight. Reliable learners vote louder.

## EDA

Understand distributions, class separation, and related features before trusting a model.

## Evaluation

Recall matters for malignant cases. Compare the same metric on the same CV setup.

**One-sentence revision: AdaBoost is a weighted vote of simple rules that repeatedly focuses on the examples it currently finds hard.**